

How to Use the Enhanced Backtesting System

Step 1: Test Everything is Working (2 minutes)

First, let's make sure the new system works without breaking anything:

```
# Test that the enhanced backtester runs  
python src/bktst_enhanced.py --help
```

```
# Test compatibility  
python src/test_compatibility.py
```

Expected: You should see help text and “ ALL COMPATIBILITY TESTS PASSED”

Step 2: Run Enhanced Backtesting (5-10 minutes)

```
# Run the enhanced backtester (same parameters as your old run.sh)  
python src/bktst_enhanced.py \  
  --start-window-days-back 120 \  
  --end-window-days-back 0 \  
  --high-frequency 1H \  
  --low-frequency 15T
```

This will: - Test the actual AdaptiveMultiStrategy (not just simple MA) - Optimize parameters for different market regimes - Create recommended_strategy.json (NOT best_strategy.json yet) - Create detailed results in adaptive_strategy_optimization.csv

Step 3: Review the Results (2 minutes)

```
# Look at what the backtester found  
cat recommended_strategy.json | python -m json.tool
```

```
# Check the detailed optimization results  
head -20 adaptive_strategy_optimization.csv
```

Look for: - total_return_pct - Should be positive - max_drawdown_pct - Should be less than 20% - regime_performance - How it performs in different markets

Step 4: Validate the Strategy (1 minute)

```
# Run validation checks
python src/validate_strategy.py
```

This checks: - Risk metrics are within safe limits - Parameters make sense - Recent performance test

If validation fails, DO NOT proceed to deployment.

Step 5: Preview Changes (1 minute)

```
# See what would change WITHOUT making changes
python src/strategy_migrator.py --dry-run

# This creates migration_dry_run.json
cat migration_dry_run.json | python -m json.tool
```

Review: - What parameters are changing - Expected performance improvement - Make sure `do_live_trades` is false (for safety)

Step 6: DECISION POINT

Option A: Use the New Parameters

If you're happy with the results:

```
# Create backup first
cp best_strategy.json best_strategy.backup.$(date +%Y%m%d_%H%M%S).json

# Apply the new configuration
python src/strategy_migrator.py

# This converts recommended_strategy.json → best_strategy.json
```

Option B: Keep Current Parameters

If you're not ready:

```
# Just continue using your current best_strategy.json
# No changes needed!
```

Step 7: Run Trading (As Normal)

```
# Run exactly as you always do
python src/tdr.py
```

```
# All your commands work the same:
# auto_trade
# resume_auto_trade
# status
# etc.
```

Optional: Enable Enhanced Diagnostics

If you want the new diagnostic commands, edit `src/tdr.py` and add these two lines near the top (after other imports):

```
from tdr_core.command_interface_enhanced import
    enhance_shell_with_diagnostics
enhance_shell_with_diagnostics(CryptoShell)
```

Then you'll have access to: - `backtest_current_params` 1 - Test current parameters on last day - `signal_analysis verbose` - See why trades are/aren't happening - `risk_metrics` - Current risk exposure - `regime_history` 24 - Recent market regime changes



Quick Reference Card

```
# Complete workflow in order:
./run_enhanced.sh                # Runs everything below
    automatically

# Or manually:
python src/bktst_enhanced.py --start-window-days-back 120
python src/validate_strategy.py      # Check if safe
python src/strategy_migrator.py --dry-run # Preview
python src/strategy_migrator.py      # Apply (if happy)
python src/tdr.py                    # Run trading
```



Important Notes

1. **Your current setup still works** - You can ignore all of this and keep using `run.sh` and `bktst.py`
2. **The main benefit** - Enhanced backtester tests your actual `AdaptiveMultiStrategy`, not just simple MA crossovers
3. **Safety first** - New config always starts with `do_live_trades: false`
4. **Rollback is easy:**

```
cp best_strategy.backup.*.json best_strategy.json
```



If Something Goes Wrong

```
# Restore your backup
ls -la best_strategy.backup.*.json # Find latest backup
cp best_strategy.backup.XXXXXXXX.json best_strategy.json

# Continue as normal
python src/tdr.py
```



What Success Looks Like

After running enhanced backtesting, you should see: - Higher expected returns than current strategy - Good performance across all market regimes (TRENDING, RANGING, VOLATILE) - Reasonable parameters (not extreme values) - Validation passes all checks



Understanding the Output Files

recommended_strategy.json

```
{
  "backtest_metadata": {
    "test_period_start": "2025-03-01",
    "test_period_end": "2025-06-30",
    "total_days": 121
  },
  "performance_metrics": {
    "total_return_pct": 15.4, // Expected return
    "sharpe_ratio": 1.2, // Risk-adjusted return
    "max_drawdown_pct": -8.5, // Worst loss period
    "win_rate": 58.3 // Percentage of winning trades
  },
  "optimal_parameters": {
    "strategy": "AdaptiveMulti",
    "short_window": 10,
    "long_window": 46
  }
}
```

adaptive_strategy_optimization.csv

Contains all tested parameter combinations and their performance. Useful for understanding which parameters work best.

migration_dry_run.json

Shows exactly what will be in your new best_strategy.json before you commit to it.

Comparing Old vs New Approach

Feature	Old (bktst.py)	New (bktst_enhanced.py)
Tests AdaptiveMultiStrategy	✗	✓
Regime-specific metrics	✗	✓
Parameter optimization	Basic	Advanced (grid search)
Risk validation	✗	✓
Safe migration	✗	✓
Output file	best_strategy.json	recommended_strategy.json

Pro Tips

1. **Run backtests regularly** - Markets change, so reoptimize monthly
2. **Compare results** - Use `compare_strategies` command to see current vs recommended
3. **Start conservative** - Test new parameters in paper trading first
4. **Monitor regime changes** - Use `regime_history` to understand market behavior
5. **Keep backups** - The migrator creates them automatically, but extra backups don't hurt

That's it! The system is designed to be safe and easy to use. Start with the enhanced backtester and see if it finds better parameters for your trading.